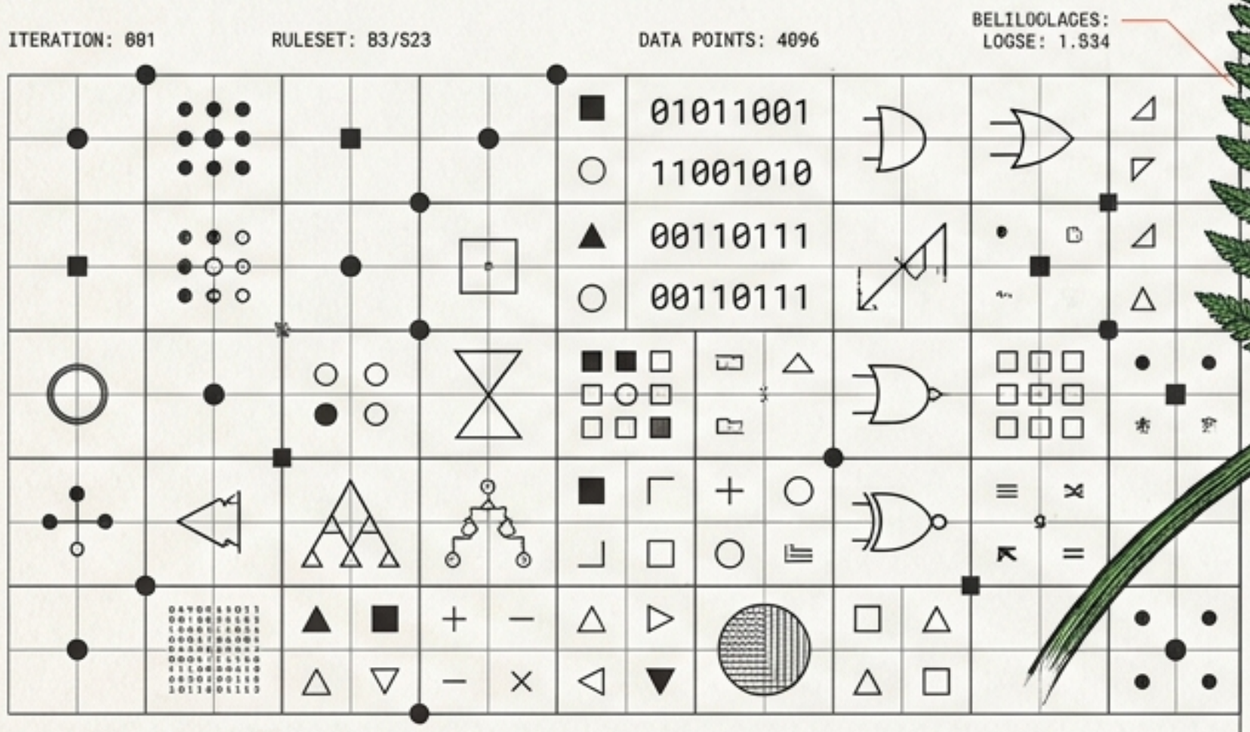
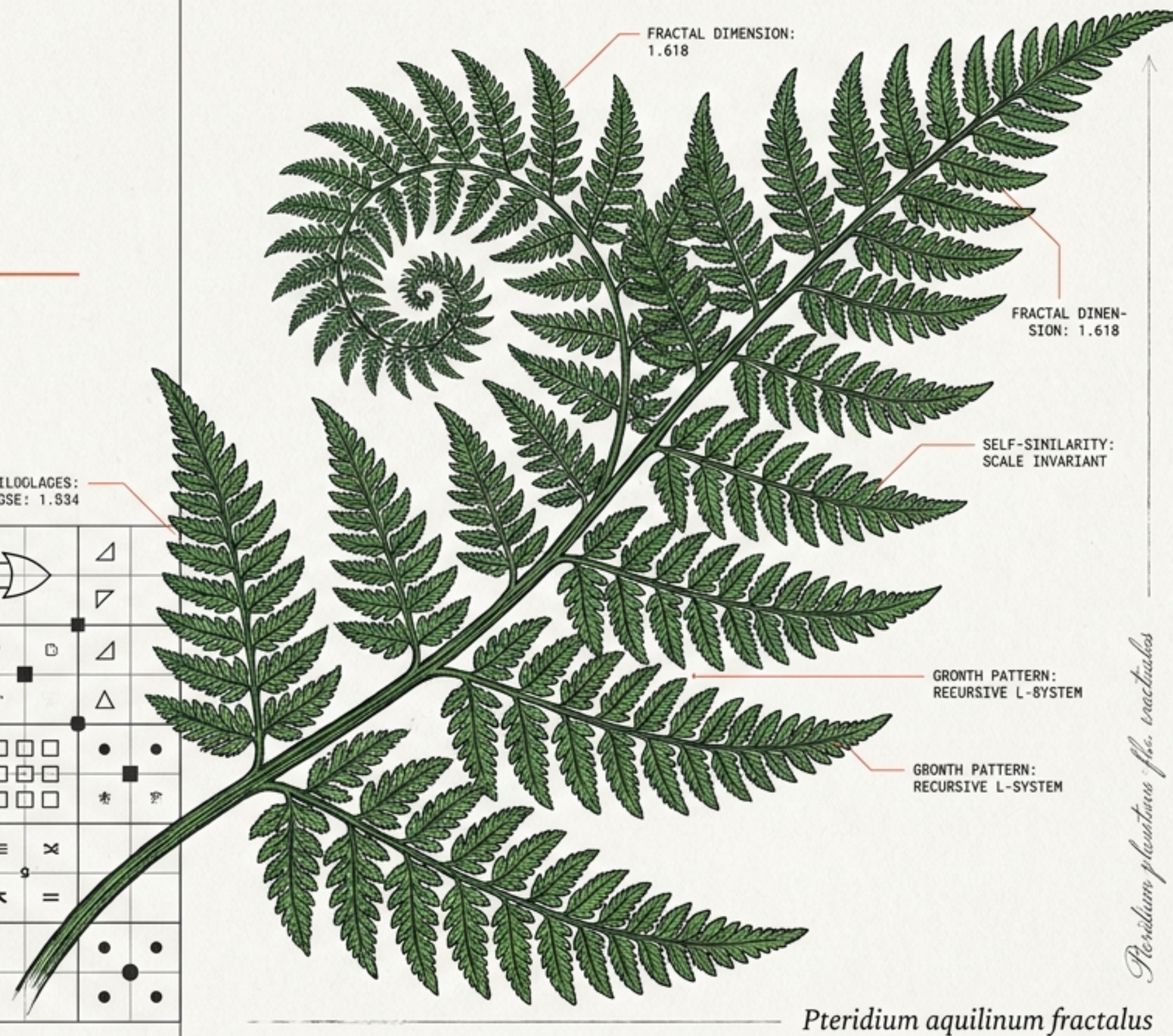
Complexitate Emergentă

Cum reguli simple creează un univers infinit.

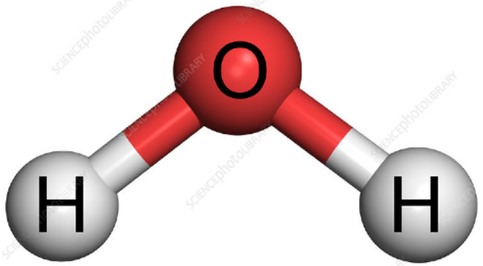




Axante Edward-Ioan

Ce este Emergența?

Conceptul de **emergență** descrie modul în care interacțiunea unor elemente simple, guvernate de reguli elementare, generează sisteme de o **complexitate** ce depășește suma părților componente.
Procesul este vizibil pretutindeni în natură, de la formarea cristalelor de zăpadă din molecule de apă până la organizarea coloniilor de furnici sau funcționarea creierului uman.



x 10^{18}



Ce au sistemele astea in comun?

în principal au doua componente:

- **Elementele sistemului** (“Cărămizile de baza”): Acestea sunt obiecte sau concepte simple care pot fi combinate sau aranjate. Pot fi elemente fizice (atomi, molecule, furnici), elemente abstracte (cuvinte, numere) sau, în informatică, simpli biți de 0 și 1.
- **Regulile:** Acestea definesc cu strictețe modul în care interacționează componentele de bază atunci când sunt combinate (de exemplu, cum se atrag, cum se resping, se suprapun sau se conectează). Regulile sunt cele care animă elementele pasive.

Exemple de emergență:

- **Apa:** O singură moleculă de H_2O nu are nimic din ceea ce numim “tensiune superficială” și nici nu arată ca un cristal, însă trilioane de molecule care urmează regulile chimiei se organizează în fulgi de zăpadă sau produc fenomene de transformare de fază (precum fierberea apei).
- **Biologia:** O furnică singură nu construiește arhitecturi ample, iar un neuron izolat nu gândește; însă o colonie de furnici și creierul uman au capacități care transcend cu totul biologia individuală a părților componente.
- **Jocurile:** În șah, piesele urmează reguli foarte simple, însă varietatea enormă a mutărilor, contra-mutărilor și strategiilor complexe explodează din aceste legi de bază. Inventatorii jocului nu au proiectat toate strategiile avansate de șah, ci acestea au “e*mers”* de la sine în timp.

Richard Dawkins

"The essence of life is not so much the atoms and small molecules that go into us as the way, the ordering, the way those molecules are put together."

Să ne imaginăm un bit el poate fi 1 sau 0 (adevărat/fals), deci 2 combinații. Pentru 2 biți avem 4 combinații, pentru 3 avem 8, pentru 10 avem 1024.

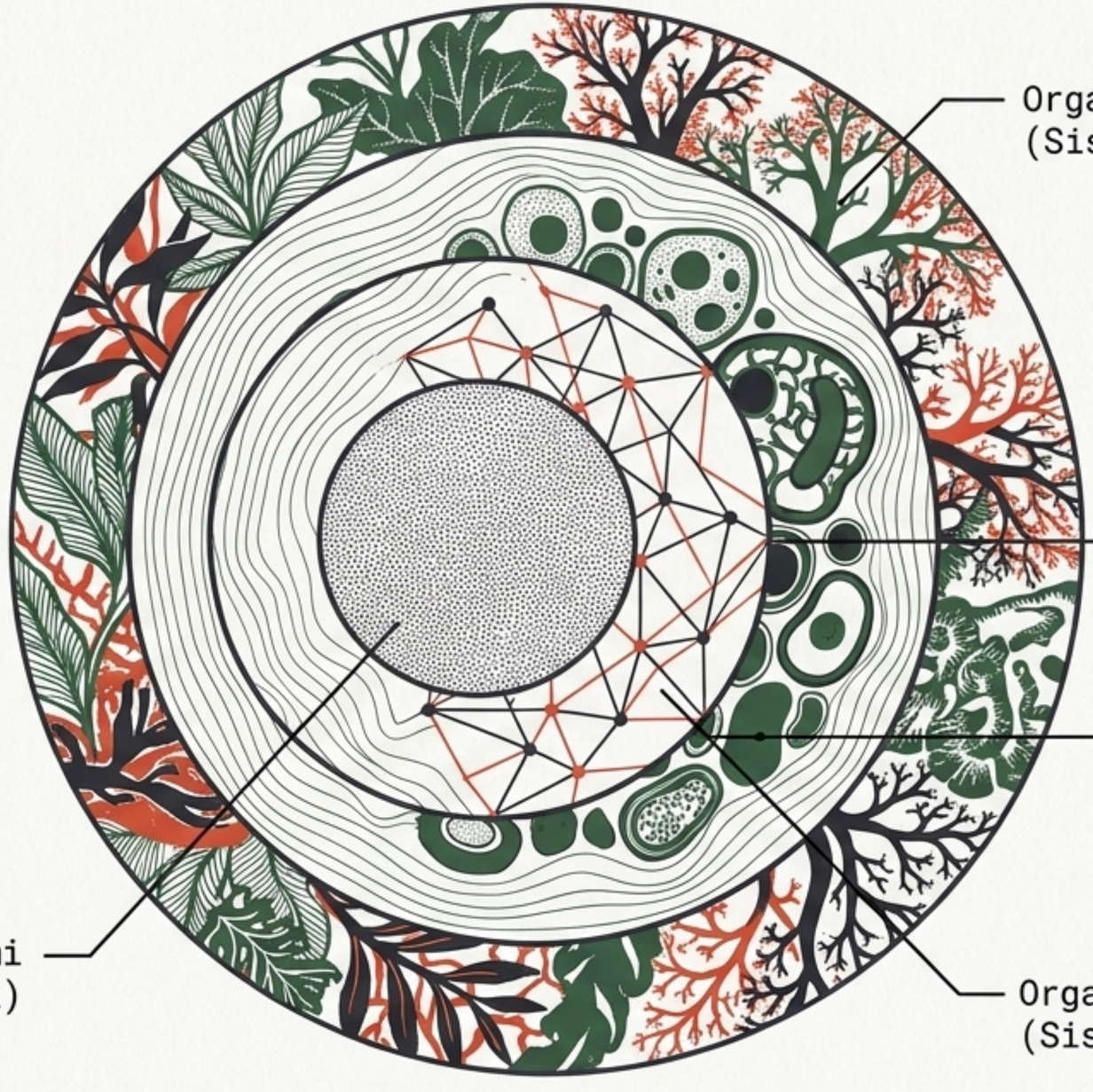
Fie n numărul de biți. Numărul total de șiruri binare unice care se pot forma este 2^n (cardinalul mulțimii părților sau spațiul stărilor).

Deci numărul de combinații posibile creste exponențial fata de numărul de biți.

Acest fenomen se numește explozie combinatorică si reprezintă “motorul” complexității emergente, prin care obținem mult mai multă informație de la sistem decât cea pe care o punem înăuntru.

Straturile Emergenței

Emergența nu se întâmplă o singură dată. Blocurile formează structuri care devin la rândul lor blocuri de bază la un nivel superior.
Fiecare strat nu este un sistem independent, ci emerge direct din stratul inferior.



Automate celulare elementare.

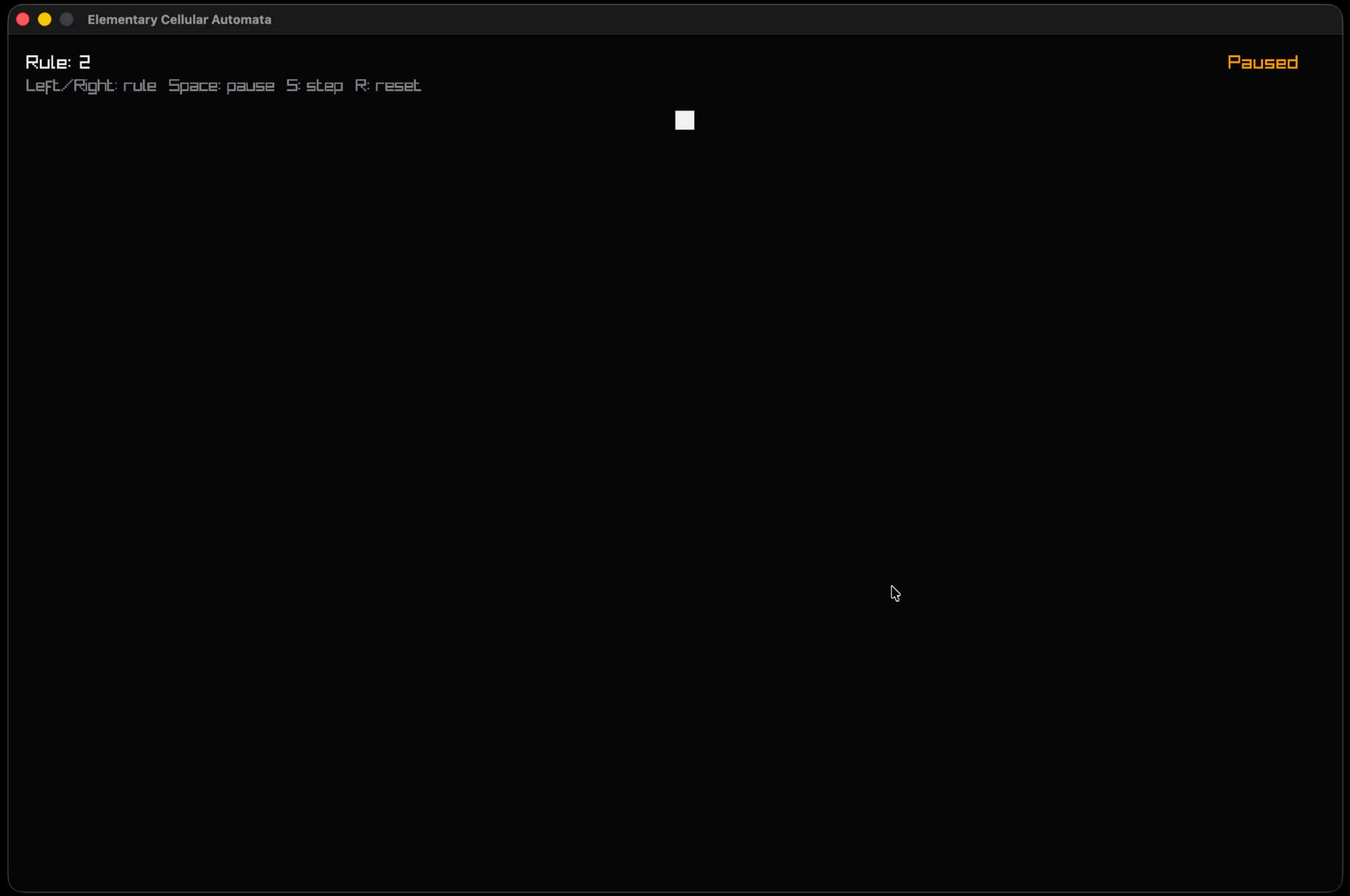
Fiecare pătrat reprezintă un automat celular cu o stare interne (1/0) := S , si un cartier := N care reprezintă celulele alăturate.

Sistemul are o funcție de tranziție $\delta(S, N) = 1/0$ care se aplica la fiecare iterație asupra fiecărei celule.

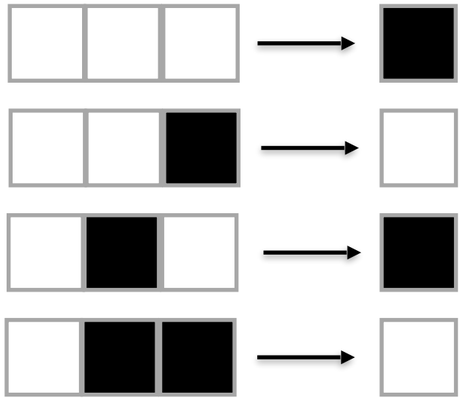


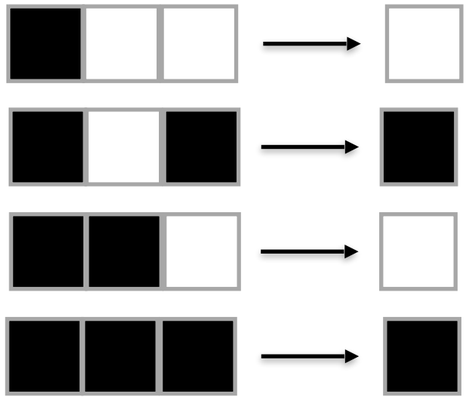


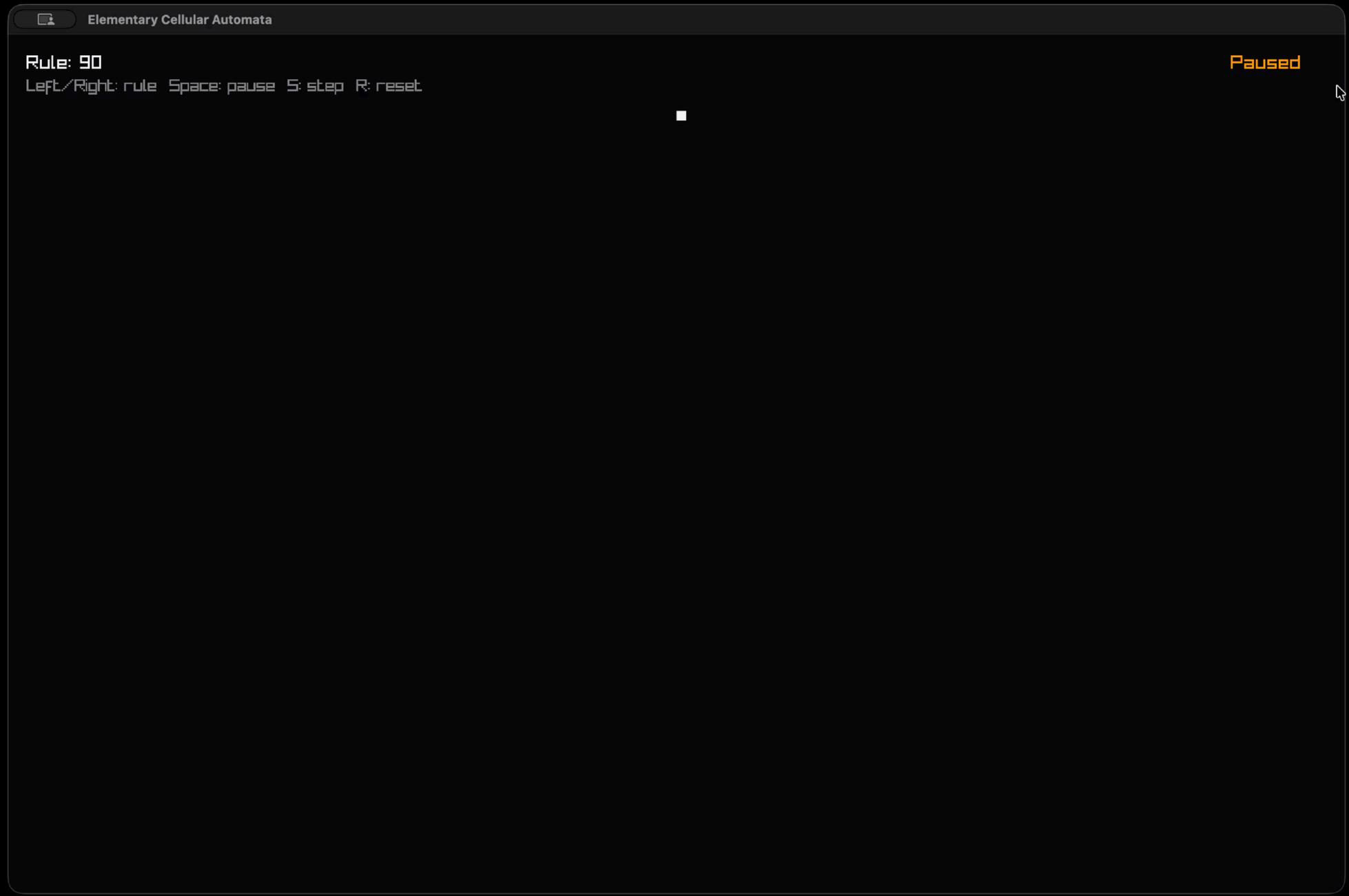
Wolfram code: Rule 2


Elementary Cellular Automata
Rule: 2
Paused
Left/Right: rule Space: pause S: step R: reset

Știți ce se va întâmplă cu acest sistem?

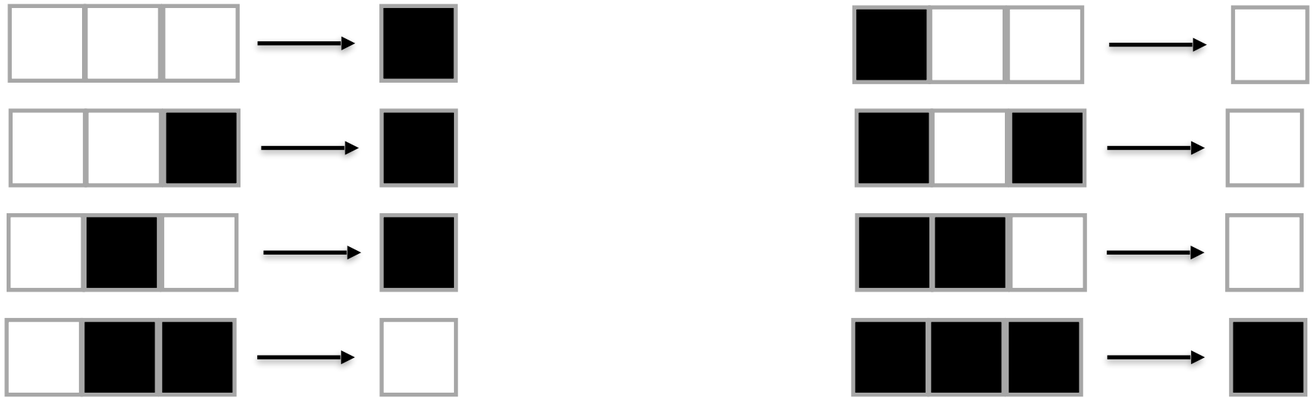


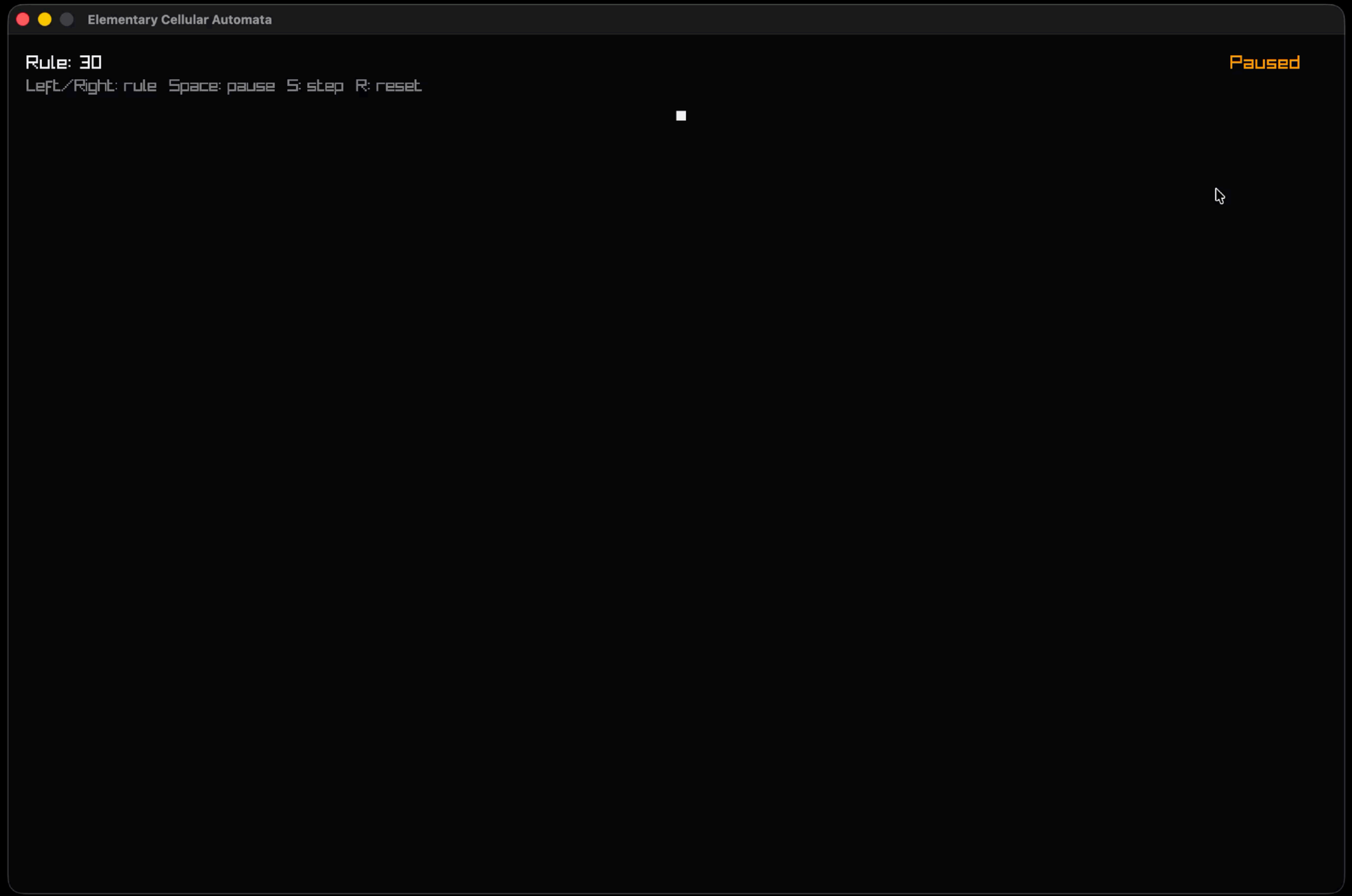




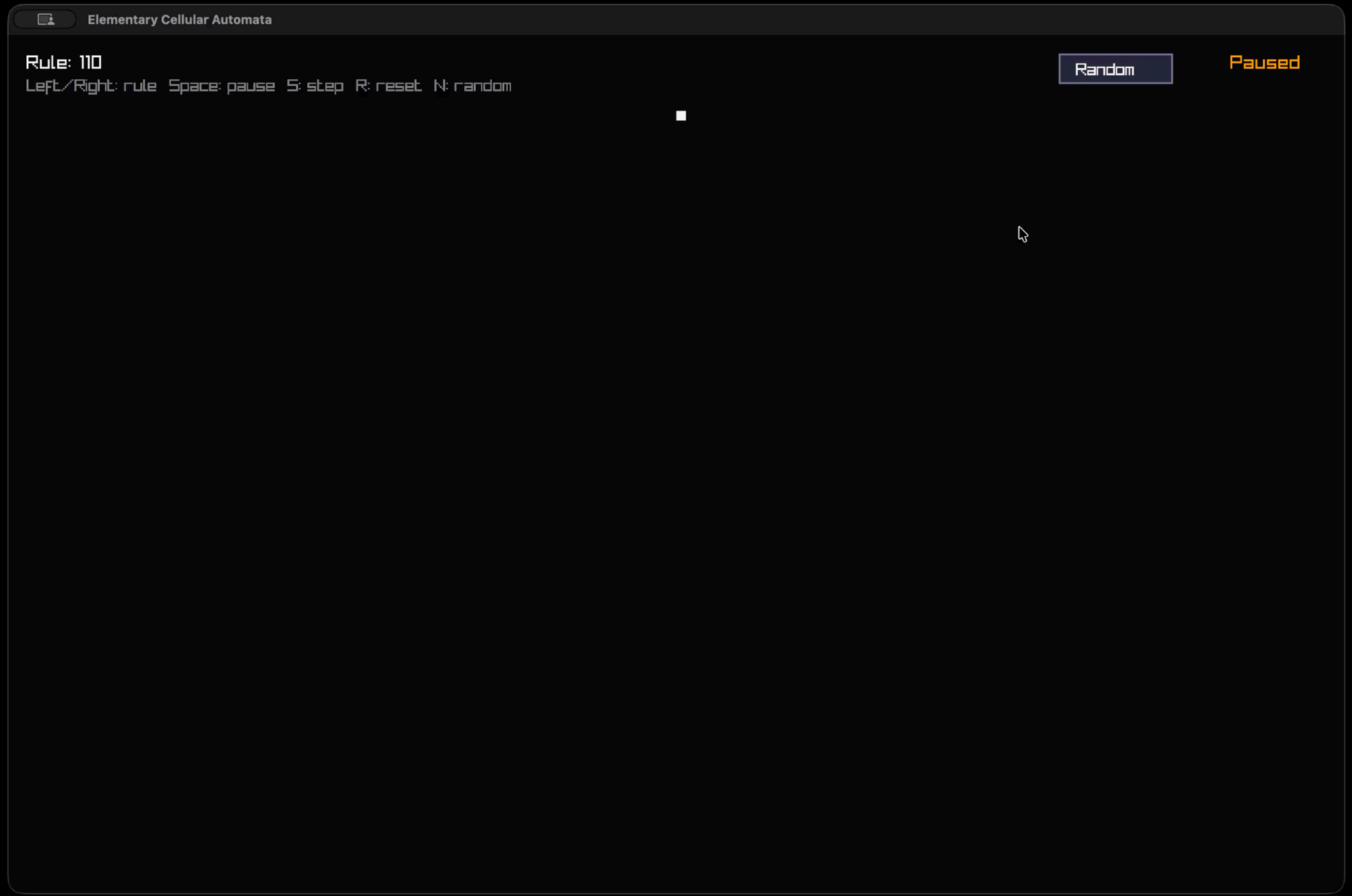
Practic este un xor Intre vecini celulei din Mjloc

Dar cu acest sistem




Elementary Cellular Automata
Rule: 30
Paused
Left/Right: rule Space: pause S: step R: reset

O complexitate remarcabilă apare atunci când aceleași reguli simple sunt aplicate repetitiv (iterativ sau recursiv) pe parcursul timpului. În programe precum automatele celulare (*Conway's Game of Life* sau *Regula 30* descoperită de Steven Wolfram), fiecare „celulă” se ghidează strict după vecinii săi imediați. Prin aplicarea acestor reguli pas cu pas, un simplu punct de pe ecran poate evolua în fractali (cum ar fi triunghiul lui Sierpinski), în forme haotice dar structurate sau în structuri auto-organizate complexe.
Game of life si Regula 110 sunt Turing complete (i. e. Pot simula orice program in interiorul lor).


Elementary Cellular Automata
Rule: 110
Left/Right: rule Space: pause S: step R: reset N: random
Random
Paused

Clasificarea lui Wolfram.

În urma analizei sale exhaustive a programelor simple de calculator (precum automatele celulare), Steven Wolfram a clasificat regulile sistemelor în **patru categorii distincte de complexitate**, pe baza tiparelor emergente pe care acestea le generează. Deși derivă din observarea codului binar, aceste categorii se aplică perfect și fenomenelor fizice din lumea reală:

1.**Plată și plictisitoare (Omogenă)** Această categorie reprezintă sisteme în care totul este la fel, omogen și nu se schimbă niciodată. Modelele sunt uniforme și lipsite de variație pe întreaga lor întindere. În lumea fizică, exemple ale acestei categorii simpleun cub de oțelul pur sau cuburile de gheață.

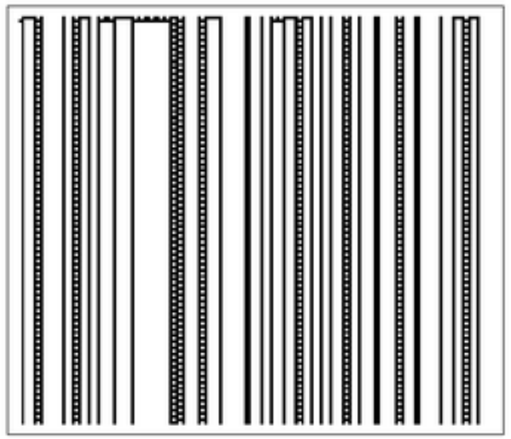
2.**Repetitivă și predictibilă** Aici întâlnim tipare care se repetă constant și al căror comportament viitor poate fi prezis cu ușurință. Această formă de complexitate implică oscilație sau o mișcare de tip "du-te-vino" regulată. Exemple din realitate sunt rocile stratificate, structurile de bază ale cristalelor, undele sonore simple și mișcarea regulată a orbitelor planetare.

3.**Haotică și aleatorie (sau pseudo-aleatorie)** A treia categorie este extrem de zgomotoasă și dezordonată, formând modele complexe care au o structură de organizare minimă spre inexistentă. Comportamentul lor este haotic, foarte greu de prezis și tinde să nu fie extrem de interesant de analizat tocmai din cauza lipsei de ordine. Ca exemplu fizic avem amestecurile haotice de particule, precum moleculele de aer dintr-o cameră sau vaporii de apă care formează un nor.

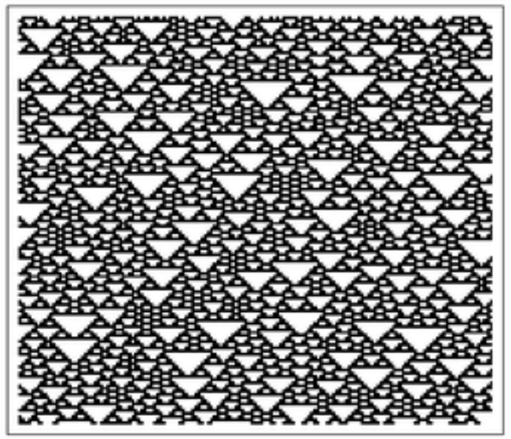
4.**Complexă și organizată** Considerată a fi cea mai fascinantă categorie, aceasta reprezintă **un amestec rafinat de ordine și haos**. Sistemele din această clasă nu sunt nici complet predictibile, dar nici total aleatorii, având deopotrivă componente structurate și dezordonate. Tiparele care se auto-organizează în acest spațiu generează cele mai interesante fenomene. Printre exemplele uimitoare se numără cristalele complexe de gheață, procesele ce dau naștere minții și vieții, și modelele vizuale de pe cochiliile melcilor conici (care seamănă extrem de mult cu faimoasa “Regulă 30” descoperită de Wolfram).



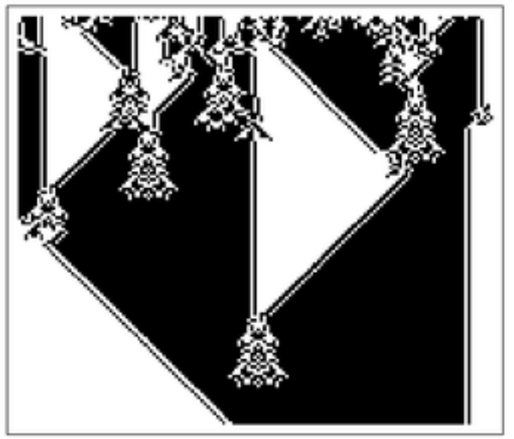
class 1



class 2



class 3



class 4

From Stephen Wolfram-A new kind of science pg.231 https://www.wolframscience.com/nks/p231--four-classes-of-behavior/

Simplitate emergentă

Conceptul de “simplitate emergentă” este o idee aparent paradoxală care sugerează că sistemul sau obiectul emergent rezultat este mai simplu decât materia fundamentală extrem de complicată din care este alcătuit.

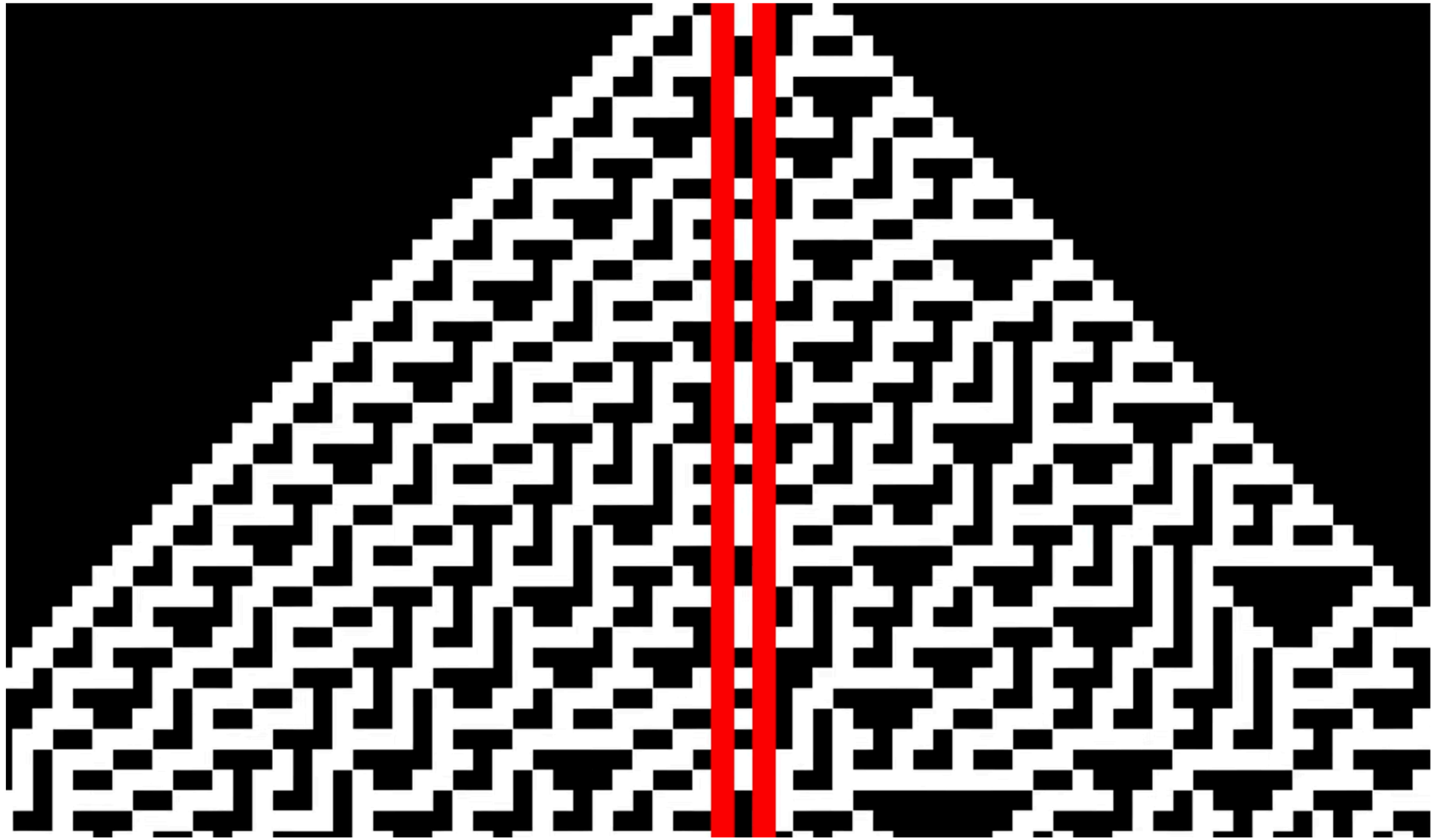
Cel mai bun exemplu pentru a ilustra acest fenomen este o **minge de basket**. La nivel microscopic, mingea este o „supă” extrem de complexă de particule, fiind formată dintr-un număr incalculabil de atomi. Cu toate acestea, la nivel macro, ea se manifestă pur și simplu ca o sferă cu un diametru, un centru de masă și câteva forțe și valori care o descriu suficient de bine pentru a-i putea prezice comportamentul viitor.

Prin urmare, **mingea de baseball este, într-un anumit sens, mai simplă decât un singur atom aflat în interiorul ei**, chiar dacă în același timp este categoric mai complexă, deoarece este alcătuită dintr-o multitudine de atomi. Această simplificare la un nivel superior este posibilă exclusiv datorită faptului că atomii sunt organizați.

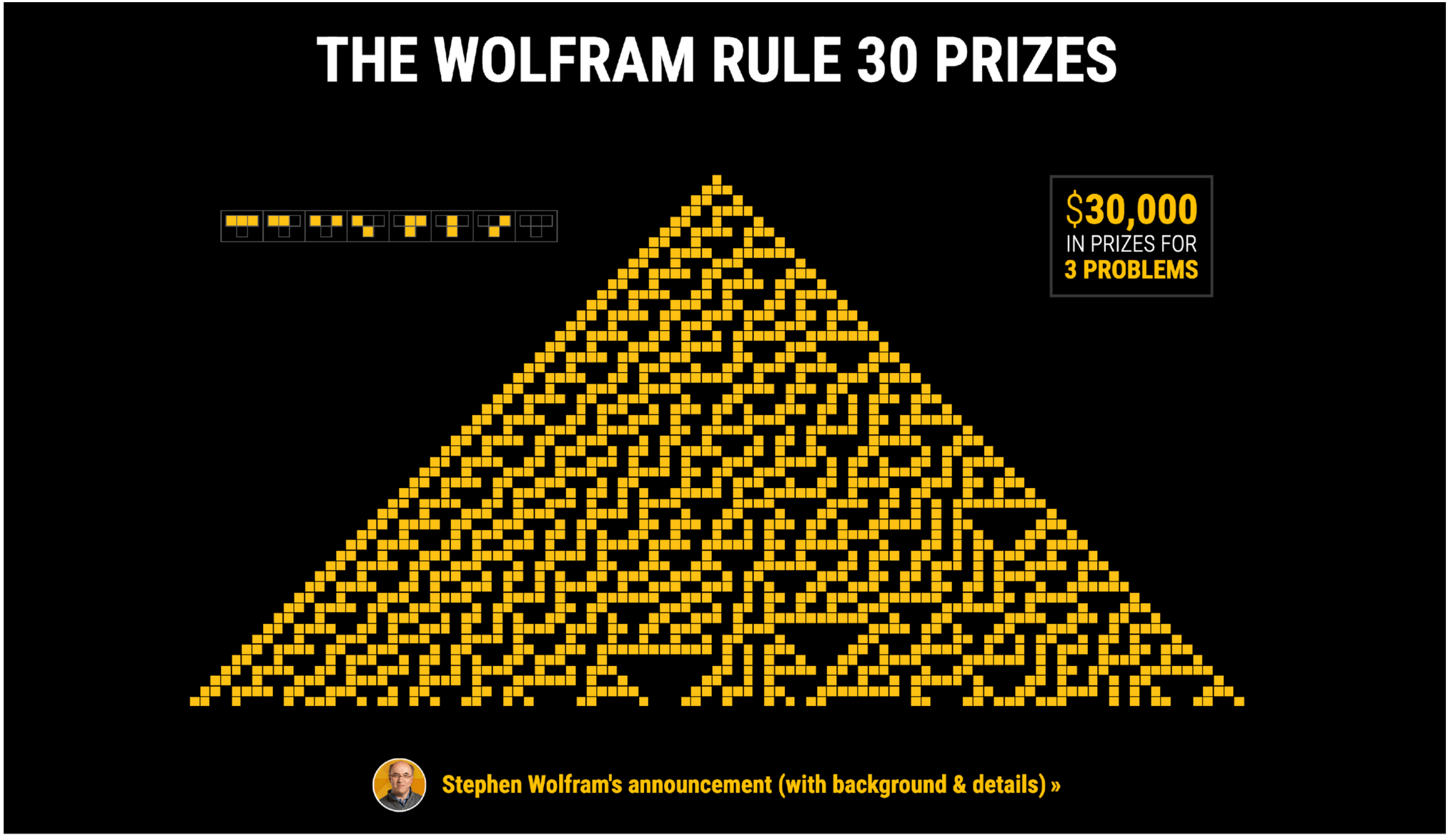
Aceste structuri locale organizate și stabile, cum ar fi o minge de baseball sau modelele de tip “glider” din game of life, sunt caracteristice sistemelor din Categoria 4 de complexitate.

Haos in sistemele emergente

Cum va fi celula din mijloc după 100 de iteratii in Rule 30?



Putem avea un algoritm o formula, un fel de scurtatura, care ne permite sa aflam rezultatul fără a itera prin toate stările pana la starea dorită?



https://rule30prize.org/

Este foarte greu sa prezici ce va face un sistem emergent chiar si când e determinist ca regula 30, va face mereu același lucru. Evident ca putem simula si itera printre stări pentru a vedea rezultatul care ne interesează, dar daca sistemul e in lumea reală, vrei o scurtatura pentru a calcula ce se va întâmplă înainte de a se întâmple.

Wolfram spune in cartea sa ca unele sisteme nu pot fi calculate cu o formula sau un algoritm si numește aceasta proprietate **ireductibilitate computaţională** (Aceasta proprietate este strâns legată in teoria compuntabilitati de Problema Opririi (Halting problem) si de existența problemelor nedecidabile)**.**

Doua motive pentru care unele sisteme sunt ireductibile computational este:

1. **Sensibilitatea extremă la condițiile inițiale (Efectul de fluture)** Chiar dacă un sistem emergent funcționează după reguli complet deterministe, predictibilitatea sa este blocată de comportamentul haotic. Sistemele sunt atât de sensibile încât o schimbare microscopică atrage consecințe imense și imprevizibile mai târziu. De exemplu, în programul *Conway's Game of Life*, modificarea valorii unei singure celule de pornire face ca tiparul final să devieze enorm față de original, afectând celule aflate la mare distanță (Butterfly effect).
2. **Amplificarea erorilor în timp** Datorită naturii haotice, simularea unui sistem nu te poate ajuta să prezici viitorul cu exactitate absolută. Chiar dacă reușești să modelezi un sistem cu o precizie de 99,99%, acea eroare infimă de 0,01% se va compune și va genera o predicție complet greșită, eroare care se va accentua cu cât încerci să prezici un viitor mai îndepărtat. Acesta este și motivul practic pentru care ne este atât de dificil să prezicem vremea.

Conceptual Wolfram evidentiaza nevoia unei noi științe care se bazează pe programe, nu pe ecuații. El vine si cu ideea ca universul nu este descris de numere ci mai degrabă printr-un algoritm si regulile sunt reprezentate de legile fizicii.

Dacă universul este un algoritm ireductibil, înseamnă că timpul este singura metodă prin care universul își află propriul viitor?


Game Of Life
Place cells, press Q to start
Cluster size: 3
Brush mode: False


Game Of Life
Place cells, press Q to start
Cluster size: 3
Brush mode: False

Alte exemple de sisteme emergente

Automate Celulare în Generarea Procedurală

O utilizare practică remarcabilă a automatelor celulare 2D (variante de *Game of Life*) este crearea hărților și a sistemelor de peșteri în game development (ex: *Minecraft*, *Terraria*).

Procesul folosește regulile locale pentru a transforma zgomotul haotic în structuri organice, stabile:

- **Pasul 1: Inițializarea (Zgomot aleatoriu):** Grila de joc este umplută la început cu biți de 0 (aer) și 1 (perete) în mod complet aleatoriu. Acest stadiu inițial face parte din **Clasa 3 a lui Wolfram (Haos pur)**.
- **Pasul 2: Filtrarea (Regula B45/S45678):** Se aplică o regulă de tipul *Day & Night*: o celulă devine sau rămâne perete doar dacă are un număr minim de vecini solizi (de exemplu, cel puțin 4 sau 5 vecini).
- **Pasul 3: Convergența către ordine:** Prin aplicarea repetată (iterativă) a acestei reguli timp de 4-5 generații, colțurile ascuțite se „netezesc”. Sistemul părăsește haosul și converge spre **Clasa 2 (Structuri stabile)**, generând coridoare și peșteri cu aspect organic.


Game Of Life
Cluster size: 3

Boids (Istoric,utilitate practica)

Boids este un program clasic de viață artificială dezvoltat de Craig Reynolds în 1986, care simulează realist comportamentul de stol al păsărilor, peștilor și roiurilor. Numele este o versiune prescurtată a “bird-oid object”.

În loc să utilizeze instrucțiuni complexe, centralizate, algoritmul boids produce mișcări de grup uimitor de fluide și sincronizate prin programarea fiecărui agent individual să urmeze trei reguli locale simple:

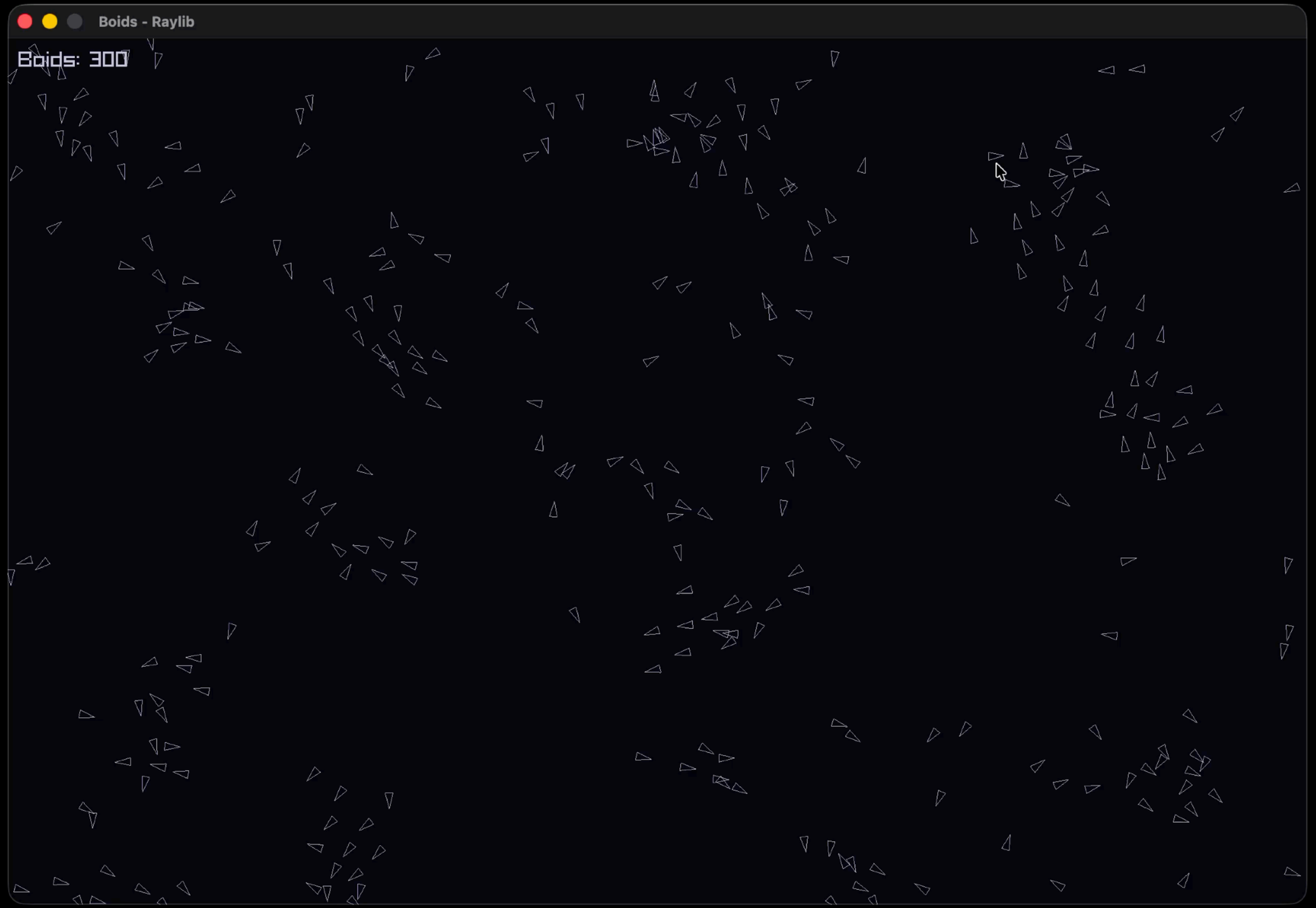
- Separare: obiectele se îndepărtează pentru a evita aglomerarea.
- Aliniere: obiectele se orientează spre direcția si viteza medie a vecinilor
- Coeziune: obiectele se deplasează spre centrul de masa mediu al vecinilor

Toate aceste funcții pot fi reprezentate prin transformări liniare
In sistemul vectorial simulat

```
Vector2 alignment(const std::vector<Boid>& boids) const {
    Vector2 sum{0.0f, 0.0f};
    int count = 0;
    for (const auto& other : boids) {
        if (!isNeighbor(other, kAlignmentRadius, true)) continue;
        sum = Vector2Add(sum, other.velocity);
        ++count;
    }

    if (count == 0) return {0.0f, 0.0f};

    Vector2 avg = Vector2Scale(sum, 1.0f / static_cast<float>(count));
    Vector2 desired = SetMag(avg, maxSpeed);
    Vector2 steer = Vector2Subtract(desired, velocity);
    return Limit(steer, maxForce);
}
```


Boids - Raylib
Boids: 300


SmoothLife - raylib
SmoothLife
[SPACE] Resume
[R] Randomize [C] Clear
Mouse Left: add mass Mouse Right: erase
Mouse Wheel: brush size [UP/DOWN]: dt
[/]: change neighborhood radii
dt 0.070 inner 2.7 outer 8.0 brush 4

În concluzie, această prezentare ne arată că cele mai uimitoare sisteme nu sunt proiectate de sus în jos, ci **se nasc direct din complexitate**.

Când am scris codul în C++ pentru această simulare de **Boids**, eu nu am programat forma finală a stolului. Nu i-am impus programului cum să deseneze acele valuri dinamice pe ecran. Am definit doar trei reguli simple, locale și deterministe: **Separare, Aliniere și Coeziune**.

Cu toate acestea, comportamentul care a urmat este guvernat de **ireductibilitatea computațională**. Nu există nicio formulă magică pe pământ capabilă să anticipeze unde va fi o anumită pasăre la pasul un milion. Singura modalitate de a afla... este să lăsăm algoritmul să ruleze.”